第八章 面向对象分析与UML建模

张宇霞

北京理工大学 计算机学院

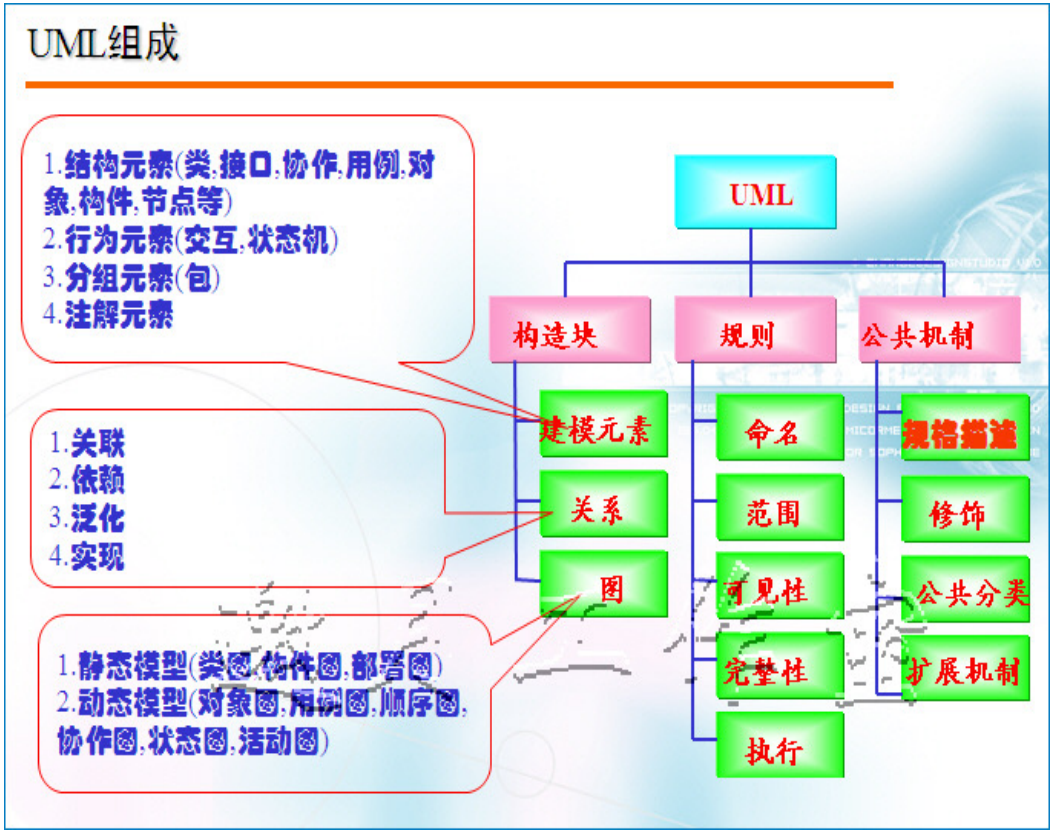
Email: yuxiazh@bit.edu.cn

面向对象分析与UML建模(2)

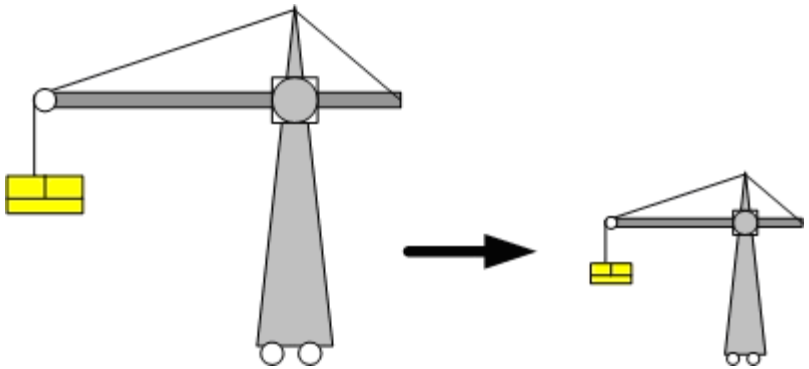
- 1. UML组成要素
  - 1.1. 基本构造块
  - 1.2. 规则
  - 1.3. 公共机制
- 2. UML模型和建模过程
- 3. 用例图

1. UML组成元素

- **基本构造块**：也就是建模元素，是模型的主体
- **UML规则**：也就是支配基本构造块如何放在一起的规则
- **公共机制**：运用于整个UML模型中的公共机制、扩展机制



UML面向对象视角下的世界



- 首先建立反应现实世界中不同事物的“构造块”，然后通过规则确定“构造块”之间的“关系”，再在公共机制下确定各个构造块的属性和“行为”。这样，在软件系统中就可以模拟现实世界的“构造块”之间的交互与协作
- 面向对象软件开发的核心思想就是高内聚（封装）、低耦合（消息驱动），使用简洁的接口拼合简单的**部件**

UML概览

- 基本构造块
  - 建模元素
    - 结构事物
      - 类
      - 主动类
      - 用况
      - 接口
      - 组件
      - 节点
      - 协作
    - 行为事物：交互 and 状态机
    - 组织事物：包
    - 注释事物
  - 关系
    - 依赖
    - 关联
    - 泛化
    - 实现
  - 图
    - use case diagram
    - class diagram
    - object diagram
    - sequence diagram
    - collaboration diagram
    - state chart diagram
    - activity diagram
    - component diagram
    - deployment diagram
- 规则
  - 名字规则
  - 范围规则
  - 可视性规则
  - 整体性规则
  - 执行规则
- 公共机制
  - 说明
  - 修饰
  - 通用划分
  - 扩展性

- 1.1. 基本构造块
  - 1.1.1. 事物构造块
  - 1.1.2. 关系构造块

1.1.1. 事物构造块

- **事物构造块**是对模型中最具有代表性的成分的抽象
  - (1) 结构事物：UML中的名词，它是模型的静态部分，描述概念或物理元素。
  - (2) 行为事物：UML中的动词，它是模型中的动态部分，是一种跨越时间、空间的行为。
  - (3) 分组事物：UML中的容器，用来组织模型，使模型更加的结构化。
  - (4) 注释事务：UML中的解释部分，和代码中的注释语句一样，是用来描述模型的。

(1) 结构事物

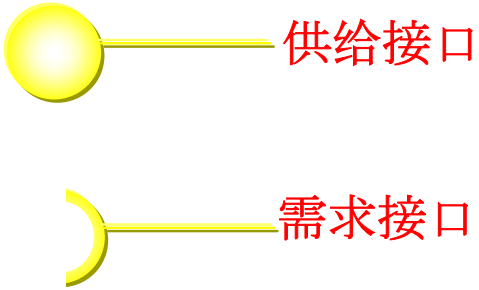
- 类（class）和对象（object）
- 接口（interface）
- 主动类（active class）
- 用例（use case）
- 协作（collaboration）
- 构件（component）
- 节点（node）

类和对象

- 类是对一组具有相同属性、相同操作、相同关系和相同语义的对象的抽象
- UML中类是用一个矩形表示的，它包含三个区域，最上面是类名、中间是类的属性、最下面是类的方法
- 对象则是类的一个实例

接口

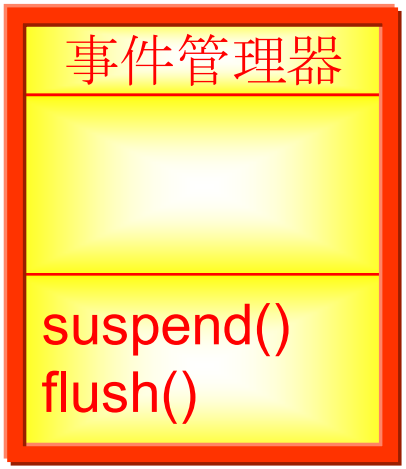
- 接口是描述某个类或构件的一个服务操作集



主动类

- 主动类实际上是一种特殊的类。引用它的原因，实际上是在开发中需要有一些类能够起到**启动控制活动**的作用。

- 主动类是指其对象至少拥有一个进程或线程，能够启动控制活动的类 。

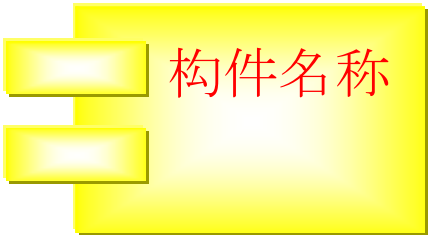


用例与协作

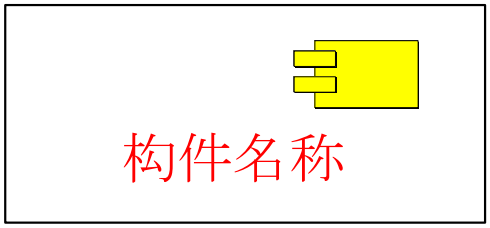
- 用例是著名的大师Ivar Jacobson首先提出的，现已经成为了面向对象软件开发中一个需求分析的最常用工具
  - 用例实例是在系统中执行的一系列动作，这些动作将生成特定执行者可见的价值结果
  - 一个用例定义一组用例实例
- 协作定义了一个交互，它是由一组共同工作以提供某协作行为的角色和其他元素构成的一个群体。
  - 对于某个用例的实现就可以表示为一个协作

构件

- 在实际的软件系统中，有许多要比“类”更大的实体，例如一个COM组件、一个DLL文件、一个JavaBeans、一个执行文件等等。为了更好地对在UML模型中对它们进行表示，就引入了构件（也译为组件）



1.0习惯用法

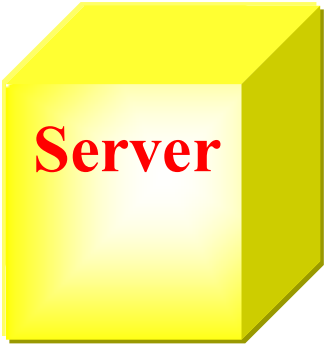


2.0习惯用法

- 构件是系统设计的一个模块化部分，它隐藏了内部的实现，对外提供了一组外部接口。在系统中满足相同接口的组件可以自由地替换

节点

- 为了能够有效地对部署的结构进行建模，UML引入了节点这一概念，它可以用来描述实际的PC机、打印机、服务器等软件运行的基础硬件



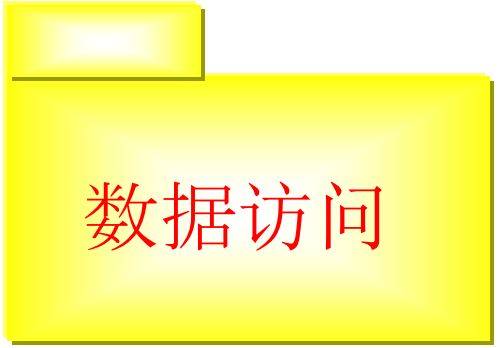
- 节点是运行时存在的**物理元素**，它表示了一种可计算的资源，通常至少有存储空间和处理能力

(2) 行为事物

- 交互（interaction）: 是在特定语境中，共同完成某个任务的一组对象之间交换的信息集合
  - 交互的表示法很简单，就是一条有向直线，并在上面标有操作名
- 状态机（state machine）：是一个对象或交互在生命周期内响应事件所经历的状态序列
  - 在UML模型中将状态画为一个圆角矩形，并在矩形内写出状态名称及其子状态

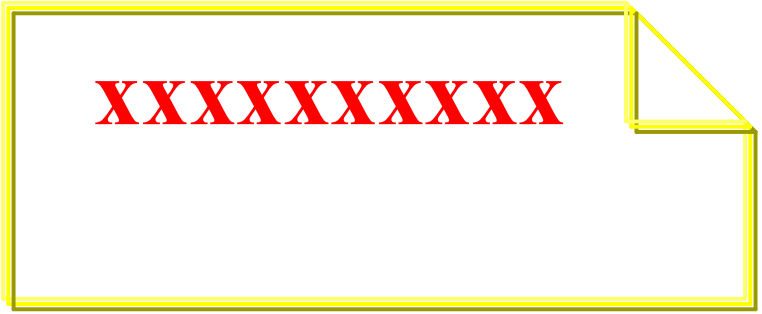
(3) 分组事物

- 对于一个中大型的软件系统而言，通常会包含大量的类，因此也就会存在大量的结构事物、行为事物，为了能够更加有效地对其进行整合，生成或简或繁、或宏观或微观的模型，就需要对其进行分组。在UML中，提供了“包（Package）”来完成这一目标



(4) 注释事物

- 结构事物是模型的主要构造块，行为事物则是补充了模型中的动态部分，分组事物而是用来更好地组织模型，似乎已经很完整了。而注释事物则是用来锦上添花的，它是用来在UML模型上添加适当的解释部分



1.1.2. 关系构造块
---UML模型关系的种类

种类	变种	表示法	关键字或符号	种类	变种	表示法	关键字或符号
抽象	派生	依赖关系	《derive》	导入	私有	依赖关系	《access》
	显现		《manifest》		公有		《import》
	实现	**实现关系**	虚线加空心三角	信息流			《flow》
	精化	依赖关系	《refine》	包含并			《merge》
	跟踪		《trace》	许可			《permit》
关联		**关联关系**	实线	协议符合			未指定
绑定		依赖关系	《bind》(参数表)	替换		依赖关系	《substitute》
部署			《deploy》	使用	调用		《call》
扩展	Extend		《extend》(扩展点)		创建		《create》
扩展	extension	**扩展关系**	实线加实心三角		实例化		《instantiate》
泛化		**泛化关系**	实线加空间三角		职责		《responsibility》
包含		依赖关系	《include》		发送		《send》

关联关系

- 关联（Association）表示两个类之间存在某种语义上的联系。关联关系提供了通信的路径，它是所有关系中最通用、语义最弱的。
- 在UML中，使用一条实线来表示关联关系
- 在关联关系中，有两种比较特殊的关系：聚合和组合

聚合　　组合

- 聚合关系：聚合（Aggregation）是一种特殊形式的关联。聚合表示类之间的关系是**整体与部分**的关系
- 如果发现“部分”类的存在，是**完全依赖**于“整体”类的，那么就应该使用“组合”关系来描述

泛化、实现、扩展与依赖

- **泛化关系**描述了一般事物与该事物中的特殊种类之间的关系，也就是父类与子类之间的关系。
- **实现关系**是用来规定接口和实现接口的类或组件之间的关系。接口是操作的集合，这些操作用于规定类或组件的服务。
- **扩展**表示将一个构造型附加到一个元类（metaclass）上，使得元类的定义中包括这个构造型。
- 有两个元素X、Y，如果修改元素X的定义可能会引起对另一个元素Y的定义的修改，则称元素Y**依赖**（Dependency）于元素X。

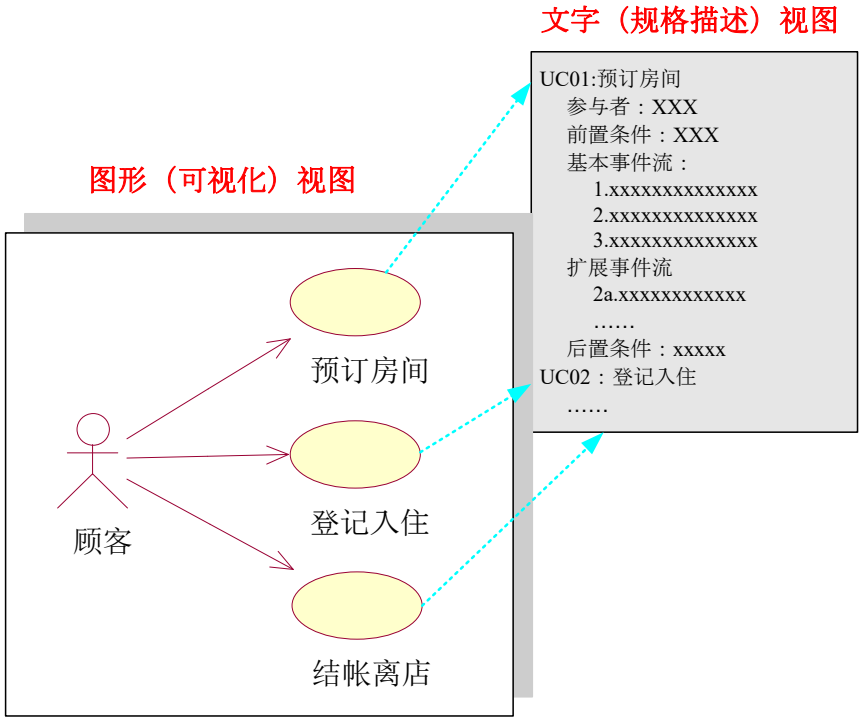
1.2. UML规则

- **命名**：也就是为事物、关系和图起名字。和任何语言一样，名字都是一个标识符
- **范围**：与类的作用域相似，包括所有者作用域（owner scope）和目标作用域（target scope）两类
- **可见性**：

可见性	规则	标准表示法	Rose 属性	Rose 方法
public	任一元素，若能访问包容器，就可以访问它	+		
protected	只有包容器中的元素或包容器的后代才能够看到它	#		
private	只有包容器中的元素才能够看得到它	–		
package	只有声明在同一个包中的元素才能够看到该元素	~		

1.3. UML公共机制：规格描述

- 在图形表示法的每个部分后面都有一个规格描述（也称为详述），它用来对构造块的语法和语义进行文字叙述。这种构思，也就使可视化视图和文字视图的分离：



1.3. UML公共机制: 修饰与通用划分

- 在为了更好的表示这些细节，UML中还提供了一些**修饰**符号，例如不同可视性的符号、用斜体字表示抽象类

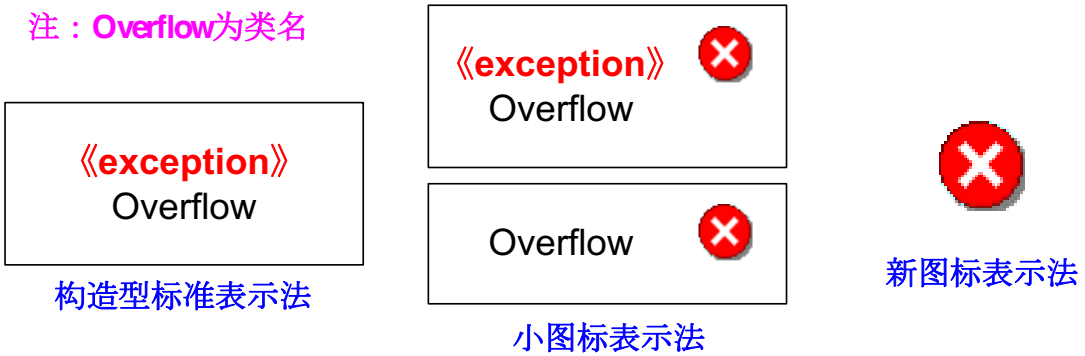
- **UML通用划分：**

  1）类与对象的划分：类是一种抽象，对象是一个具体的实例

  2）接口与实现的分离：接口是一种声明、是一个契约，也是服务的入口；实现则是负责实施接口提供的契约

1.3. UML扩展机制

- 构造型：在实际的建模过程中，可能会需要定义一些特定于某个领域或某个系统的构造块



- 标记值则是用来为事物添加新特性的。标记值的表示方法是用形如“{标记信息}”的字符串
- 约束是用来增加新的语义或改变已存在规则的一种机制。约束的表示法和标记值法类似，都是使用花括号括起来的串来表示，不过它是不能够放在元素中的，而是放在相关的元素附近

面向对象分析与UML建模(2)

- 1. UML组成要素
  - 1.1. 基本构造块
  - 1.2. 规则
  - 1.3. 公共机制
- 2. UML模型和建模过程
- 3. 用例图

UML定义的图

图名	功能	备注
类图	描述类、类的特性以及类之间的关系	UML 1原有
对象图	描述一个时间点上系统中各个对象的一个快照	UML 1非正式图
复合结构图	描述类的运行时刻的分解	UML 2.0新增
构件图	描述构件的结构与连接	UML 1原有
部署图	描述在各个节点上的部署	UML 1原有
包图	描述编译时的层次结构	UML中非正式图
用例图	描述用户与系统如何交互	UML 1原有
活动图	描述过程行为与并行行为	UML 1原有
状态机图	描述事件如何改变对象生命周期	UML 1原有
顺序图	描述对象之间的交互，重点在强调顺序	UML 1原有
通信图	描述对象之间的交互，重点在于连接	UML 1中的协作图
定时图	描述对象之间的交互，重点在于定时	UML 2.0 新增
交互概观图	是一种顺序图与活动图的混合	UML 2.0新增

UML模型划分

- 用例模型Use Case Model
  - 描述使用系统功能的角色和系统相关的功能，是需求建模的重要工具。
- 静态模型（领域模型）
  - 类图、包图
- 动态模型（行为模型）
  - 顺序图、协作图、状态转换图、活动图
- 物理实现

静态模型---领域建模

- 类图（class diagram ）
  - 类图是系统模型的基础，描述系统的静态结构
  - 对象层：描述系统实体以及承载的系统责任
  - 特征层：描述实体抽象的特征
  - 关系层：实体类的固有关系

- 包图（package diagram ）
  - 描述系统的组织模型，为控制表示的复杂性

动态模型---行为建模

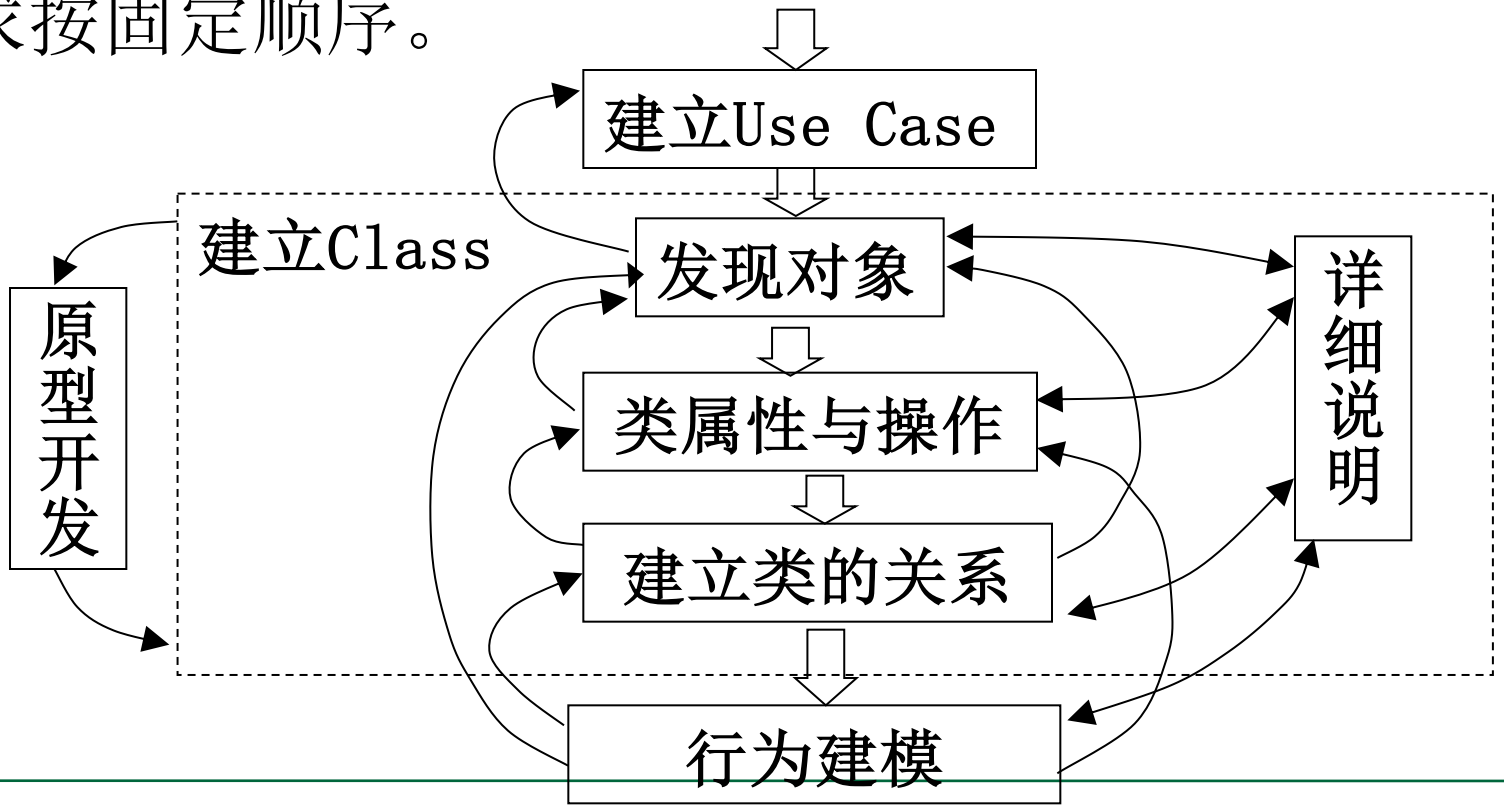
- 顺序图（sequence diagram ）
  - 描述按时间顺序排列的对象交互。
- 协作图（collaboration diagram ）
  - 表示交互对象的行为组织结构。
- 状态转换图（state chart diagram ）
  - 描述对象在生命周期内，响应事件的状态转换过程，以及响应事件后所做的反映。
- 活动图（activity diagram ）
  - 用来描述任务流程或算法过程，可用来分析系统并发事务流程

物理实现

- 部署图（deployment diagram ）
  - 用来描述系统中计算结点的拓扑结构，一个系统只有一个部署图，可用来分析分布式系统
- 构件图（component diagram ）
  - 描述一组构件以及相互间的关系，是系统实现的物理建模

UML建模过程

面向对象的建模过程，目前没有统一标准，各种建模方法的建模过程存在着较大差别，但基本原则是统一的。强调过程中各个步骤的相对独立，不要求按固定顺序。



建模过程的建议

1）首先进行需求确定

2）建立类图的过程可随时切换到其他活动

3）行为建模与类图交错进行

4）详细说明分散在各活动中

5）对详细说明要审查和补充

5）原型可以反复地进行

6）规模较小系统可以省略包图

面向对象分析与UML建模(2)

- 1. UML组成要素
  - 1.1. 基本构造块
  - 1.2. 规则
  - 1.3. 公共机制
- 2. UML模型和建模过程
- 3. 用例图

Use Case用例图

- 作用：
  - Use Case用于对系统的功能及与系统进行交互的外部事物建模
- 目的：
  - 通过寻找与系统交互的外部事物，说明他们与系统如何交互，**可以使用户和开发者，对系统的理解达成共识**

Use case 图元素

参与者：使用系统相关功能的角色

用例：与参与者有交互的一个功能

参与者与用例的关系：表示双向，没有箭头

《include》

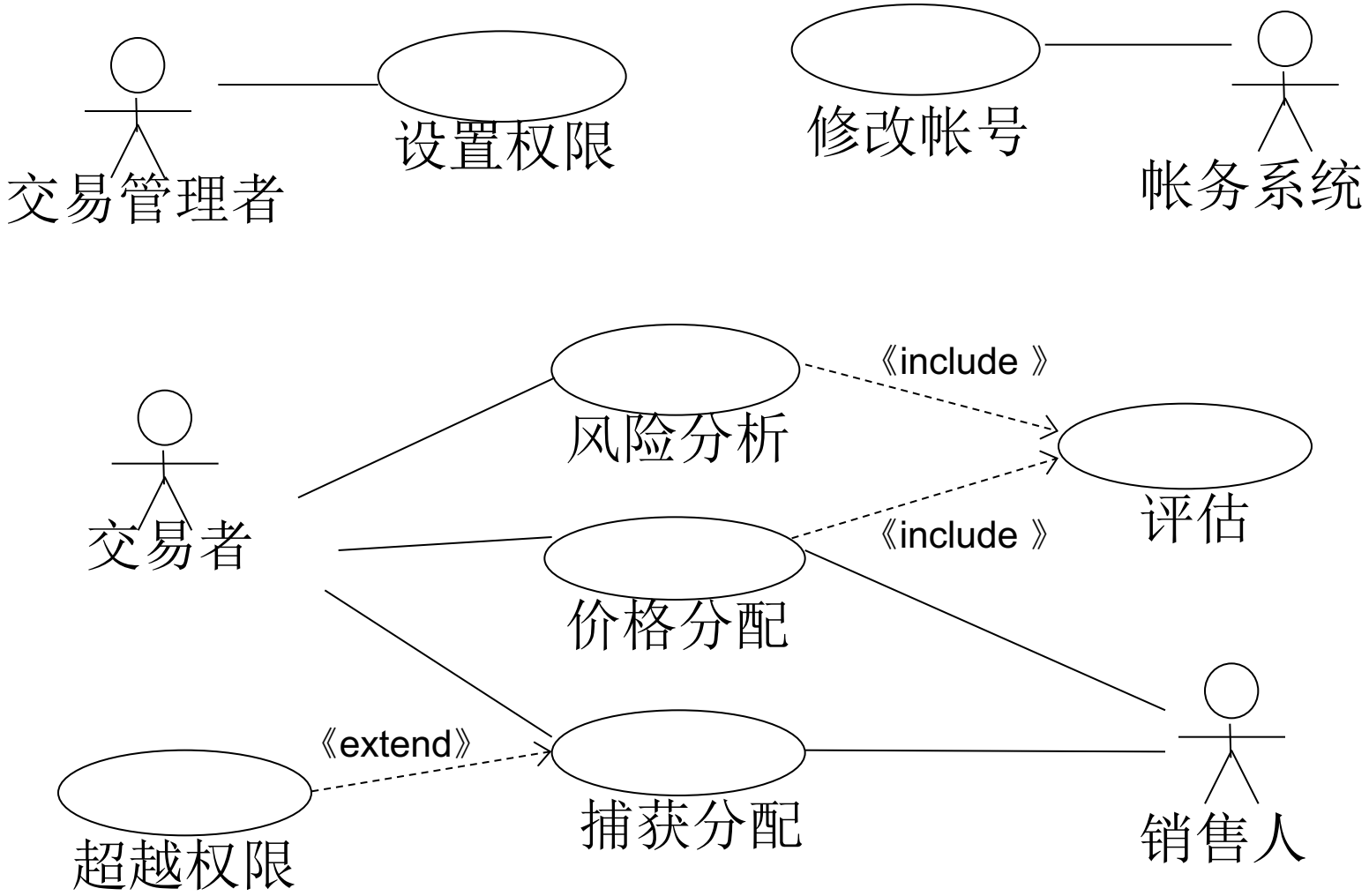
用例之间的关系：表示包含其中必须的功能

《extend》

用例之间的关系：表示扩展可选的功能

参与者之间或用例之间的关系：表示继承

Use Case示例


交易管理者
设置权限
修改帐号
帐务系统
交易者
风险分析
《include 》
评估
《include 》
价格分配
《extend》
超越权限
捕获分配
销售人

Use Case目的

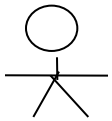
- 目标
  - 从系统边界入手，对未知空间描述
  - 系统边界---系统与外界事物的分界
  - 未知空间---待建造的未来系统
- 建立Use case的切入点
  - 与未来系统进行交互的事物
  - 包括：人员、设备、系统，以及操作功能

参与者

使用一组密切相关功能的角色。当用这组功能与系统交互来完成某项事务时，该参与者就扮演了这个的角色

例：

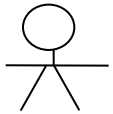
检查商品
验证顾客信用卡
收银



收款员角色

例：

查询机票信息
输入旅客信息
出票

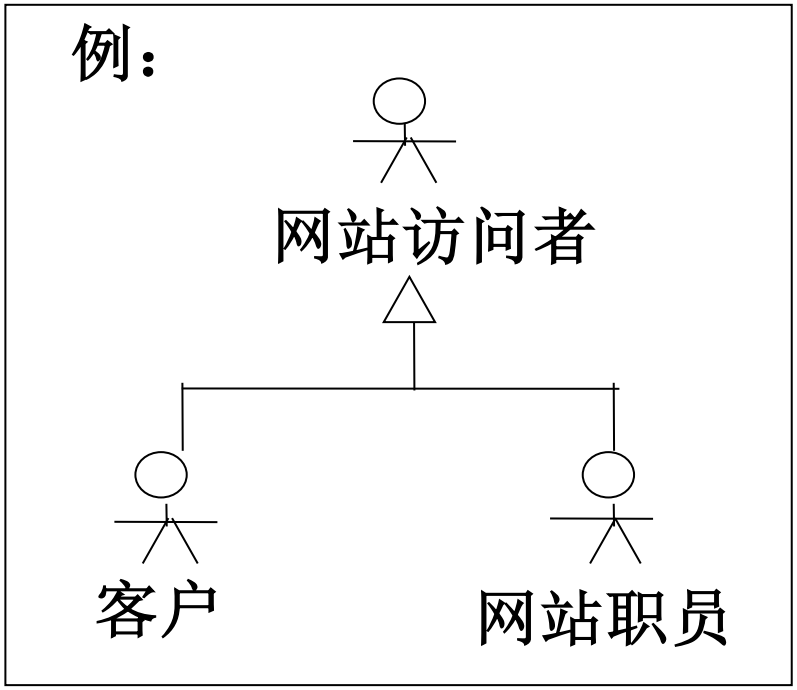


售票员角色

- 参与者可以向系统提出服务请求
- 参与者也必须接受系统的要求并做出响应
- 模型中的参与者属于系统之外的事物

参与者泛化关系

某参与者具有其他参与者的共同性质，则该参与者与其他参与者构成泛化关系，其共性可以被其他参与者继承。



表示：客户和网站职员都是网站访问者，即都有网站访问者的性质。

识别参与者

从系统的人员、设备和外部系统三个方面考虑。

注意：设备不包括显示器、键盘、鼠标这类标准接口设备，而是指计算机系统之外的系统使用设备，例如：传感器、受控马达等。

识别参与者的指导性策略：

- 谁是系统的操作者？
- 怎样使用系统？
- 系统的责任有哪些？
- 哪些参与者具有共同的行为？

用例（case）

一个用例描述系统的一项功能，该项功能可被描述为参与者可视的一组操作，其中的每个操作表示参与者与系统的一个交互过程。

用例要点：

- 用例描述系统外部可见的功能需求
- 只描述做什么，不描述怎么做
- 多数是由参与者发起的动作
- 也允许系统发起的动作，例如：异常情况处理

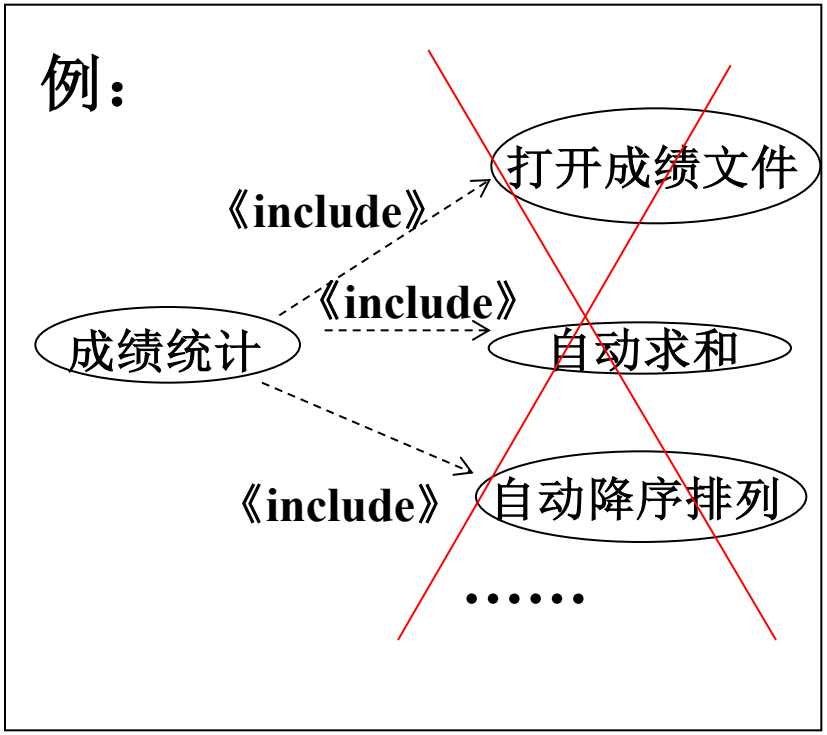
识别用例

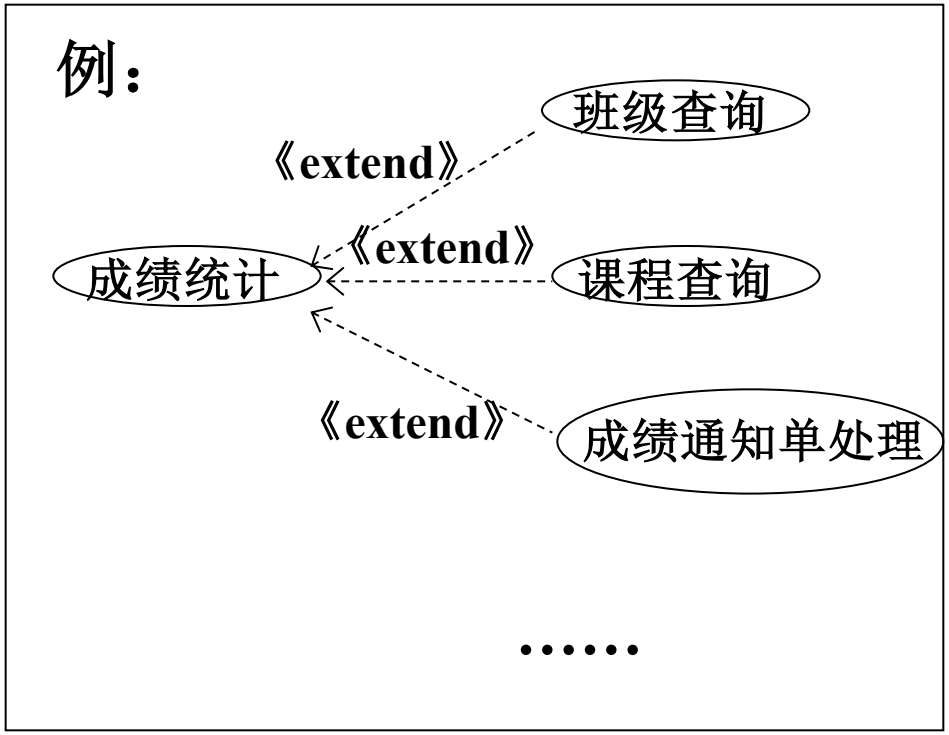
识别用例的基本方法是交互，把自己当作外部参与者，与目标系统进行交互。

考虑：

- 我和系统交互的目的是什么？
- 我需要向系统输入什么信息？
- 希望由系统进行什么处理？
- 我要从它得到何种结果？

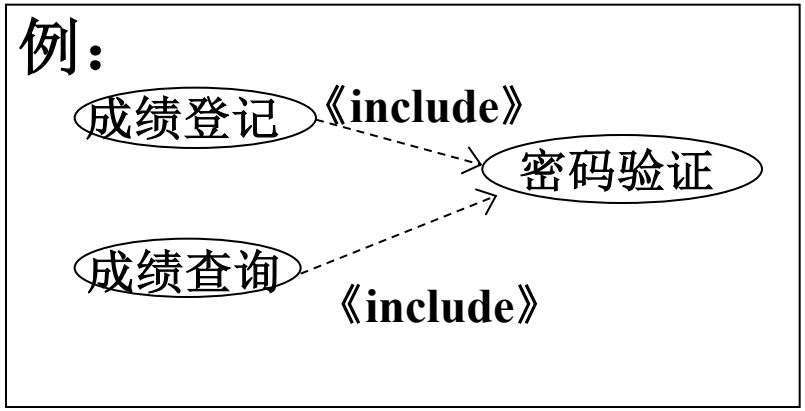
- 用例不描述功能实现的各项处理
- 用例仅描述外部可视的交互操作

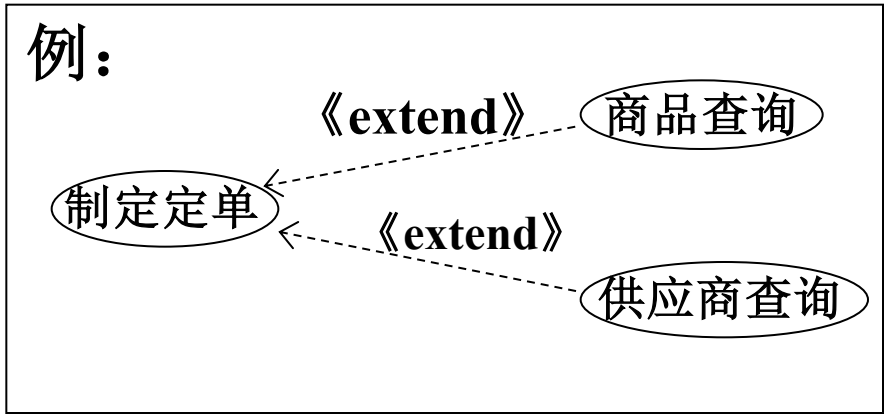


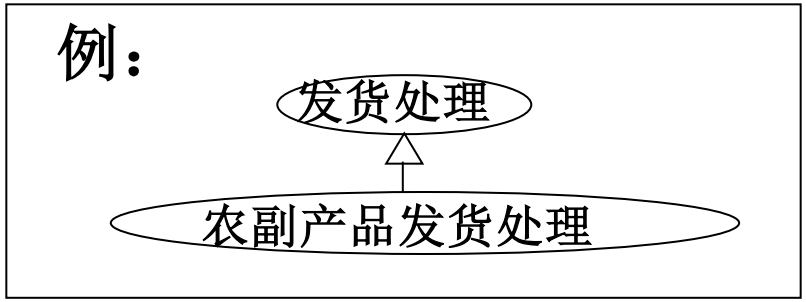


用例关系

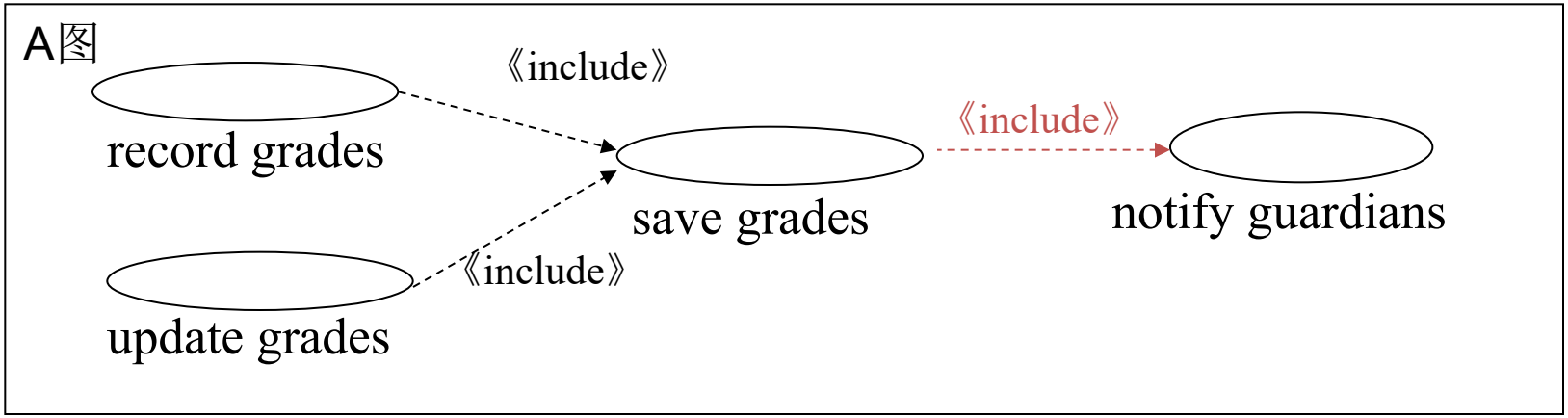
- 包含关系《include》：描述用例间**具有的公用行为**
- 扩展关系《extend》：描述用例间**可选的独立行为**
- 泛化关系 generalization：用例之间的继承关系



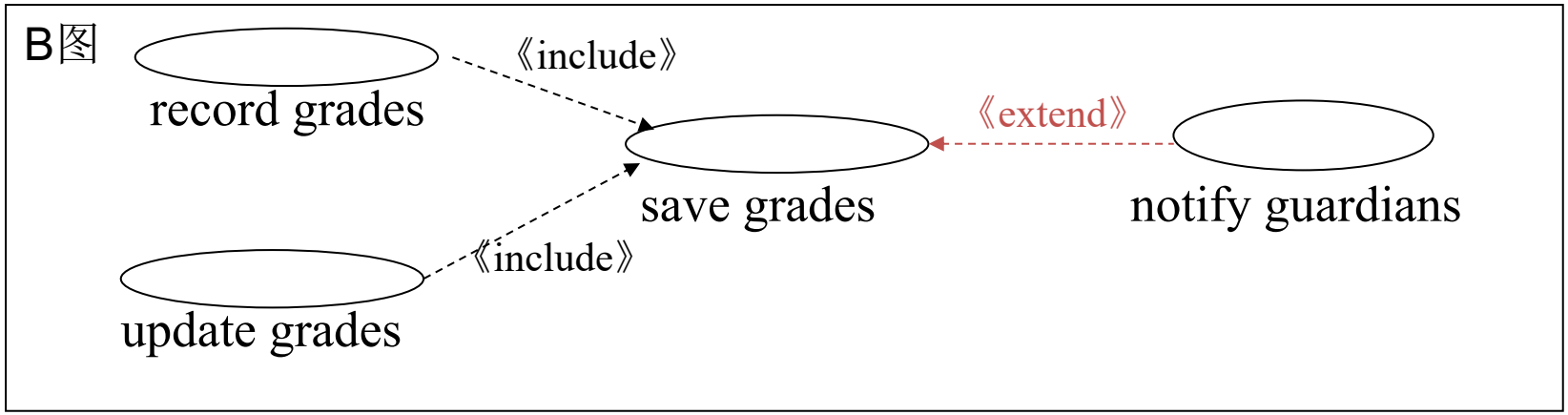




用例包含和扩展关系比较

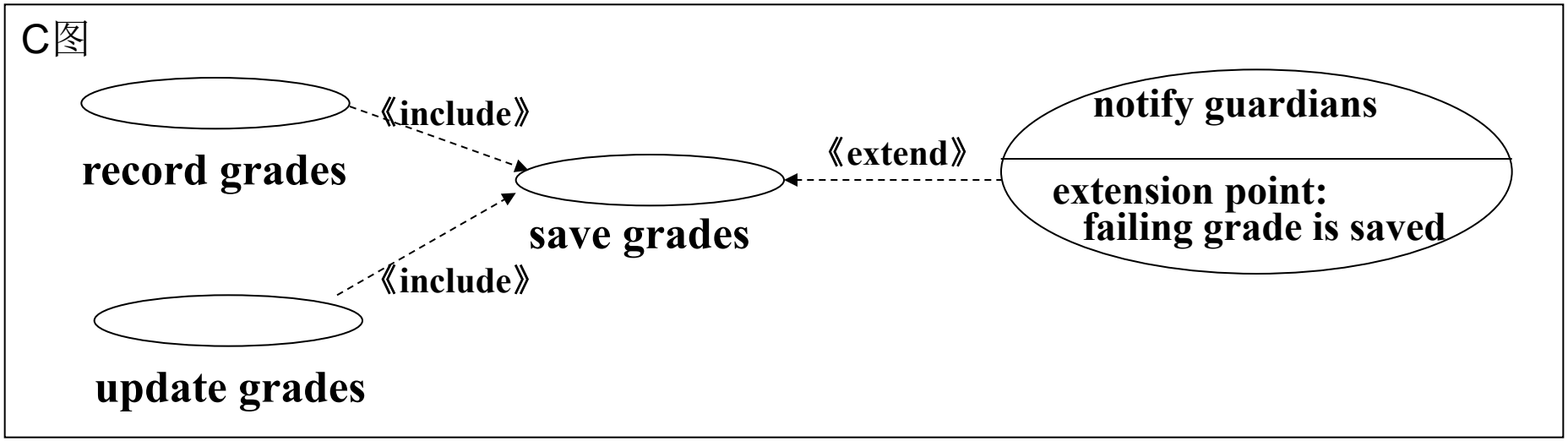


A图表示：在保存成绩之后，一定都通知监护人



B图表示：在保存成绩之后，可以选择通知监护人

Use case的扩展示例



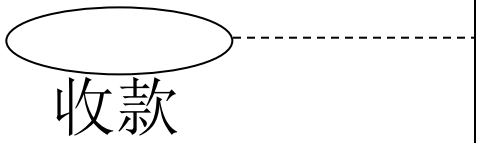
C图表示：

在保存成绩之后，可以选择通知监护人，其选择的条件是成绩不及格。（有些工具不支持扩展点图形）

用例说明

对有必要说明算法的用例，可以给出详细的说明。

例：



收款
输入本次收款的命令；
for顾客选则商品
输入商品号；
if 选择商品多于一件
商品数量+1
end if
检索商品名称及单价
减商品存量
if 商品存量低于下限
告警商品存量不足
end if
……

捕获用例

参与者的责任是基础，用例是由参与者操作的

从参与者角度分析以下问题：

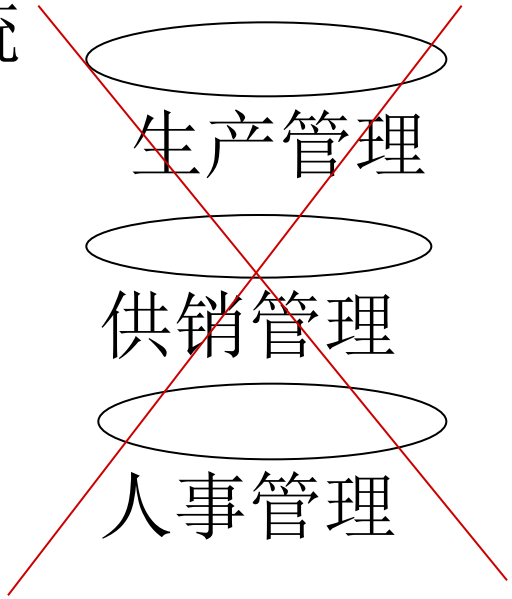
- 参与者使用系统的主要目的是什么
- 参与者使用系统所进行的各项独立事务
- 参与者怎样使用系统的服务
- 本质上不同的各项活动过程有哪些
- 除参与者外引起与系统交互有哪些

捕获用例原则

1）一个用例描述一个功能，但用例的功能不能太笼统，界定的标准是外部的参与者是否为多个角色。

太笼统的系统功能划分

企业信息系统



生产计划人员
车间管理人员

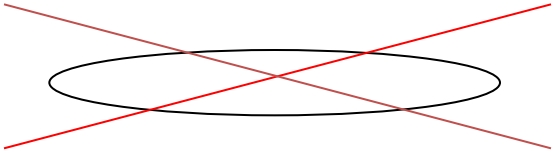
采购、计划人员
仓库管理人员
质量检验人员

工资考勤管理人员
人才招聘管理人员
离退休管理人员
临时合同人员管理

捕获用例原则

2）一个用例是在一个相对完整的时间段中发生的，应尽量避免一个用例涉及多个时间段。

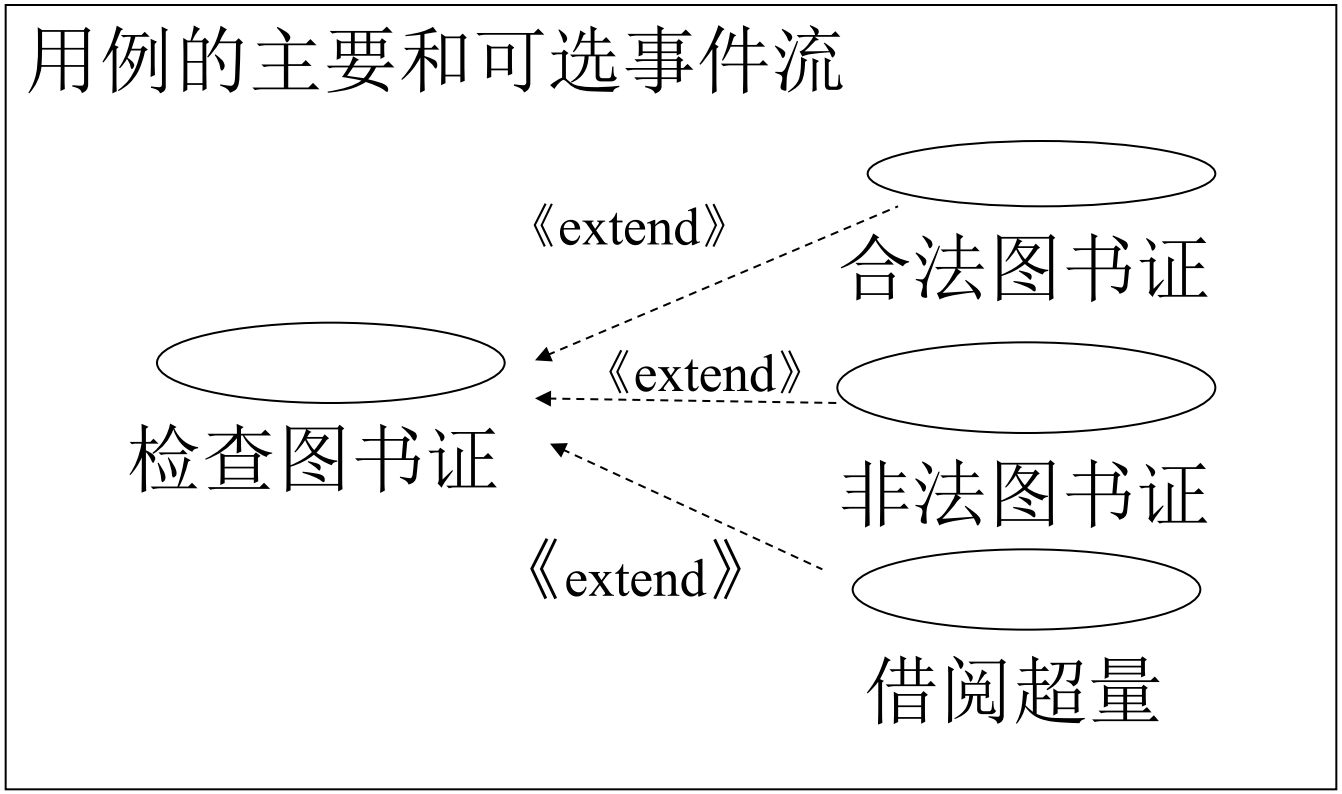
不在一个时间段的用例



订货与退货管理

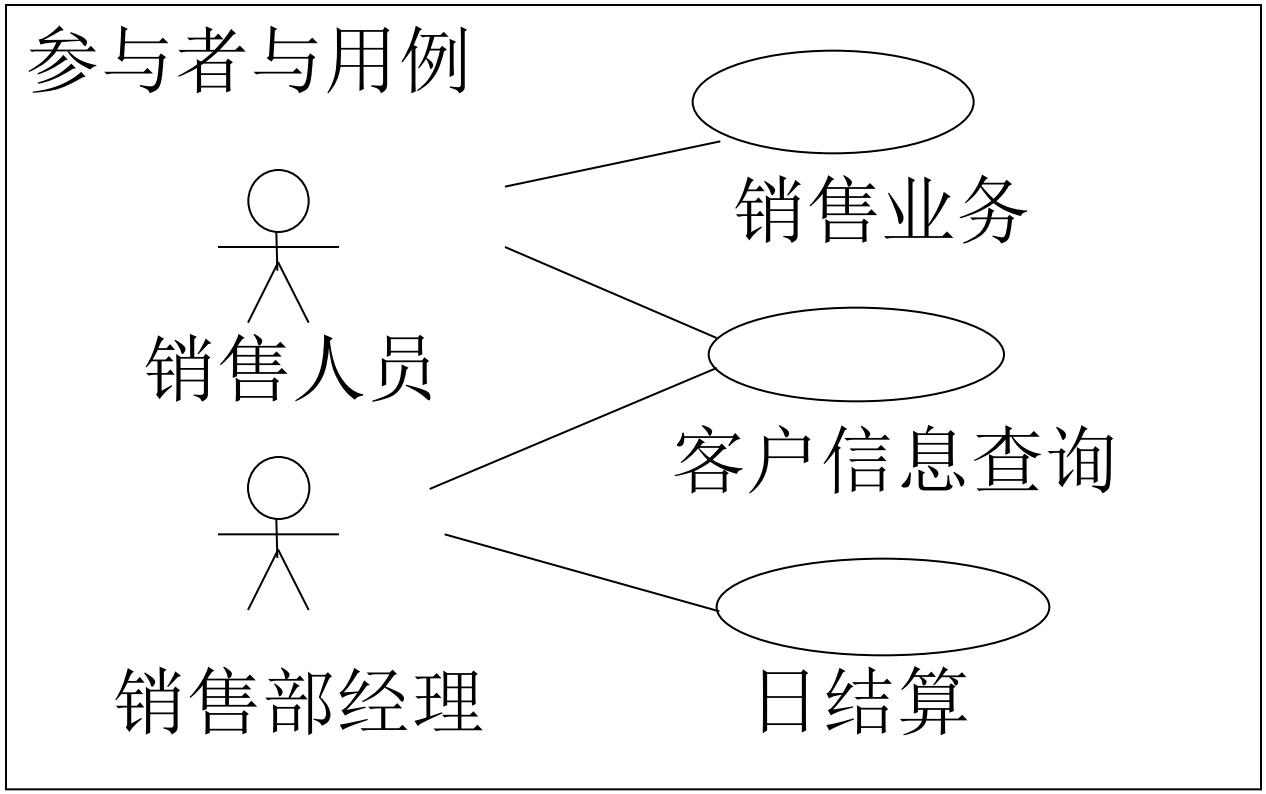
捕获用例原则

3）用例分为主要事件流与可选事件流，



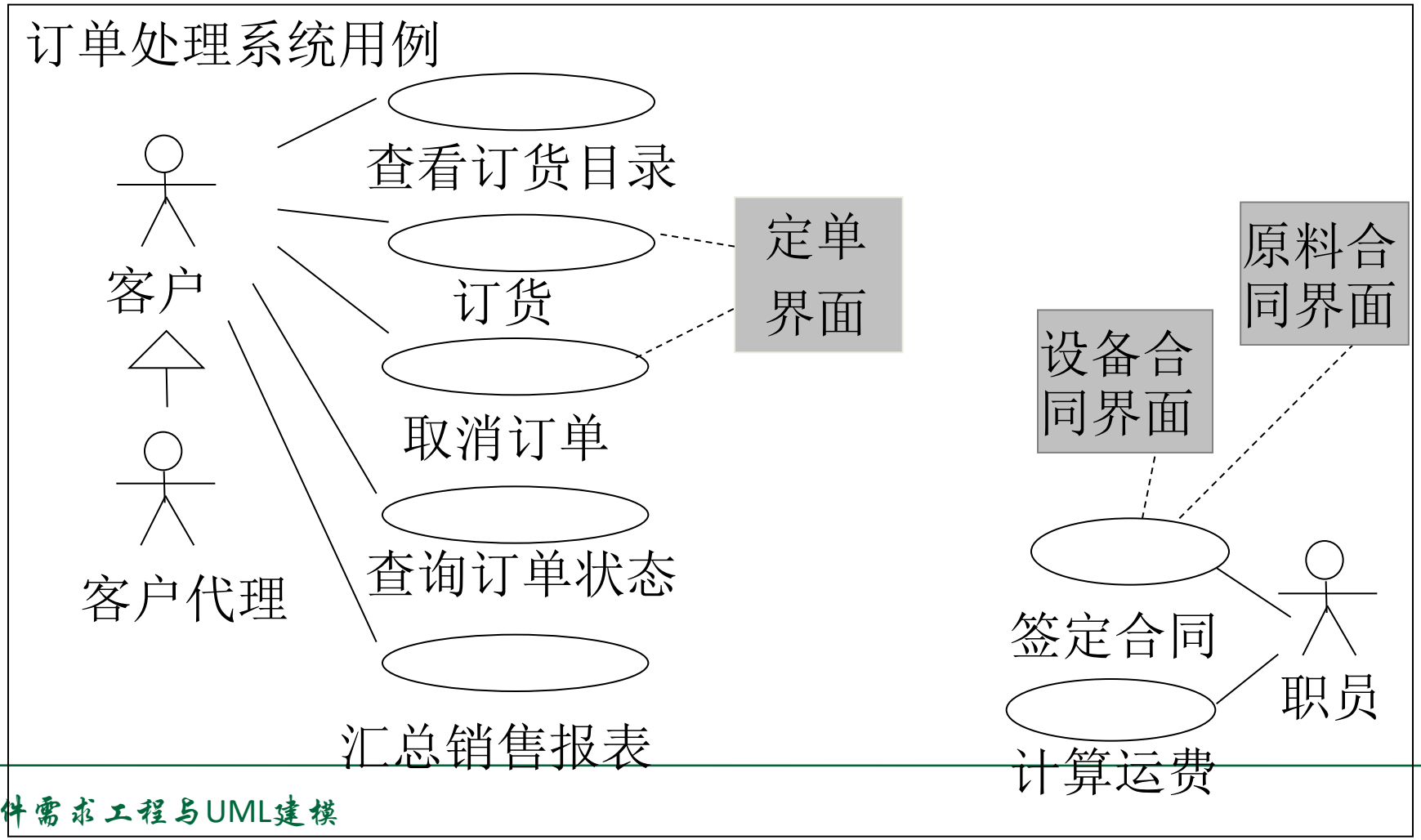
捕获用例原则

4）参与者可以对应多个用例，用例也可以对应多个参与者。



捕获用例原则

5）用例不是界面，界面也不是用例。一个用例可以对应多个界面，一个界面也可能由多个用例使用。

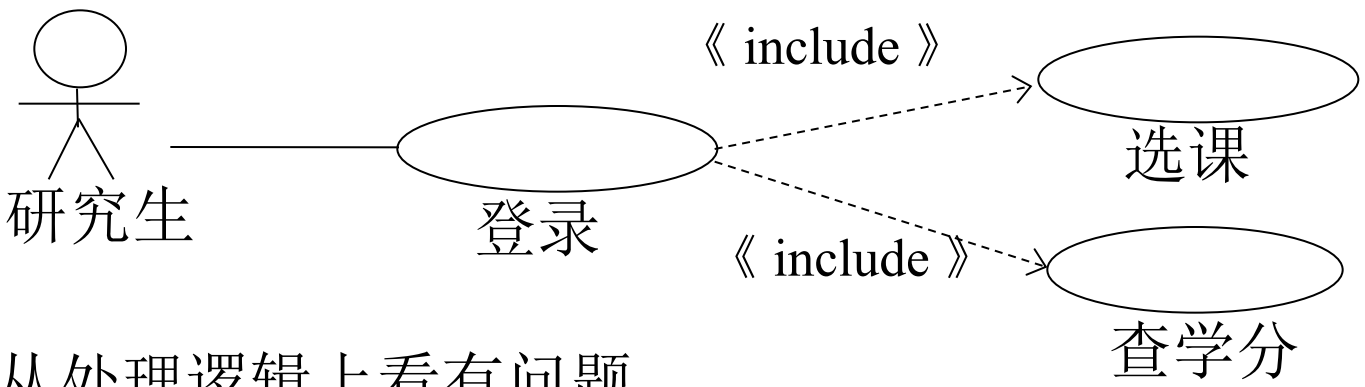


用例表示举例

研究生教务系统：对登录、选课，以及查学分功能用例描述的四种表示，说明了四种不同的工作方式。

表示一：

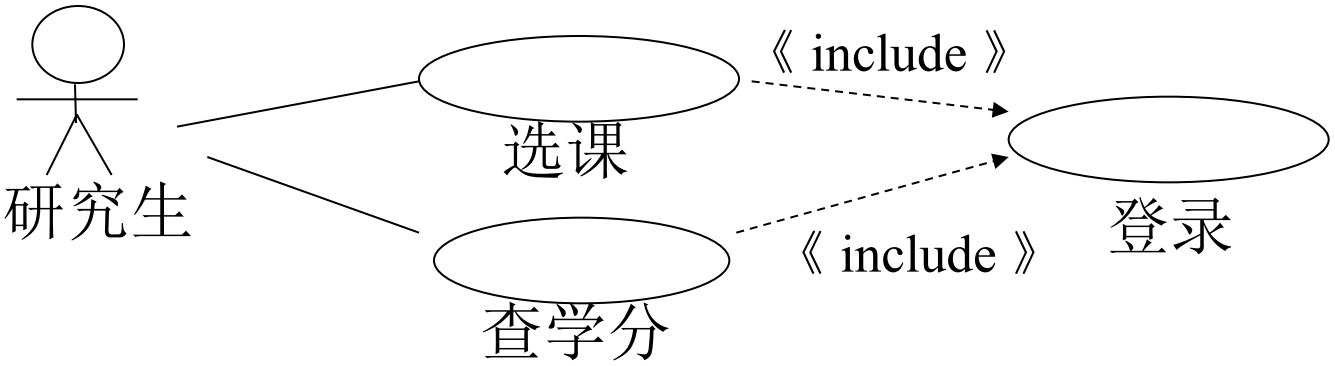
说明研究生在登录后，有两个功能是被反复使用的。两个功能作为登录主程序从属的，并且都是必须要执行的功能。



从处理逻辑上看有问题。

表示二：

研究生可选择的两个功能，都包含有登录事务的需要。每个功能的进入都必须登录。



当有这种严格的限定时，才需要这样的考虑。

表示三：

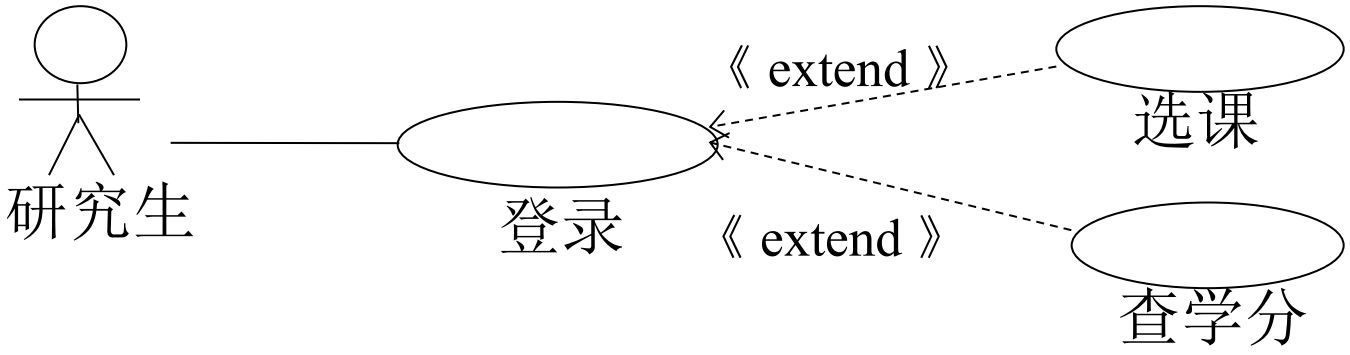
研究生在登录后，选择“选课”或“查学分”

在登录中根据必要的条件，选择是哪个功能。

条件：

if 选择“选课” then 选课

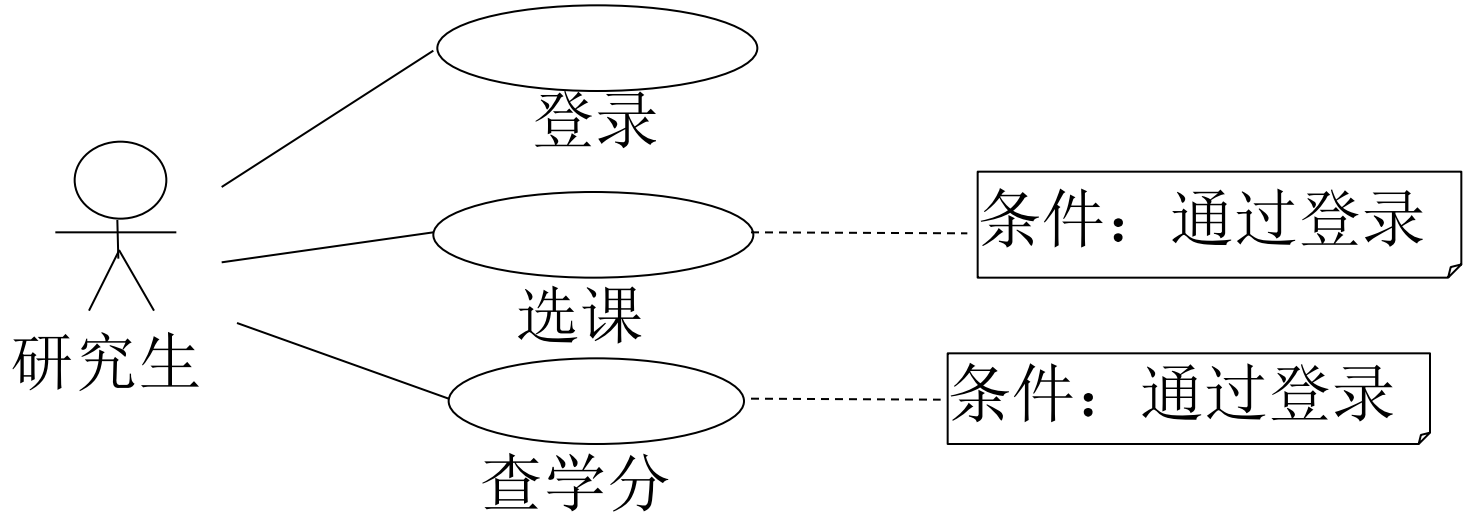
If 选择“查学分” then 查学分



若增加功能，必须修改登录的条件和判断逻辑。

表示四：

研究生有三个独立的功能，它们之间没有直接的关系，修改或添加功能，对其它的功能没有直接影响。



用例图建模的步骤

（1）找出系统外部的参与者和外部系统，确定系统的边界和范围；
（2）确定每一个参与者所期望的系统行为；
（3）把这些系统行为命名为 Use Case – 系统行为；
（4）使用泛化、包含、扩展等关系梳理出系统行为的公共或变更部分；
（5）为每个用例撰写交互流程脚本；
（6）绘制用例图；
（7）区分主事件流和异常情况的事件流，把表示异常情况的事件流作独立的用例处理；
（8）细化用例图，解决用例间的重复与冲突问题。

用例描述模板

用例编号	[为用例制定一个唯一的编号，通常格式为UCxx]	
用例名称	[应为一个动词短语，让读者一目了然地知道用例的目标]	
用例概述	[用例的目标，一个概要性的描述]	
范围	[用例的设计范围]	
主参与者	[该用例的主Actor，在此列出名称，并简要的描述它]	
次要参与者	[该用例的次要Actor，在此列出名称，并简要的描述它]	
项目相关人利益说明	项目相关人	利益
	[项目相关人员名称]	[从该用例获取的利益]
	……	……
前置条件	[即启动该用例所应该满足的条件。]	
后置条件	[即该用例完成之后，将执行什么动作。]	
成功保证	[描述当前目标完成后，环境变化情况。]	
基本事件流	步骤	活动
	1	[在这里写出触发事件到目标完成以及清除的步骤。]
	2	……(其中可以包含子事件流，以子事件流编号来表示)
扩展事件流	1a	[1a表示是对1的扩展，其中应说明条件和活动]
	1b	……(其中可以包含子事件流，以子事件流编号来表示)
子事件流	[对多次重复的事件流可以定义为子事件流，这也是抽取被包含用例的地方。]	
规则与约束	[对该用例实现时需要考虑的业务规则、非功能需求、设计约束等]	

本节小结

- 首先阐述了事物构造块、关系构造块，它们是UML建模元素的主体。
  - 事务构造块包括结构、行为、分组和注释四种类型；
  - 关系构造块主要包括关联、泛化、依赖、实现
  - 简要阐述了UML中公共的规则
- 简要地阐述了UML 2.0中定义的13种图，以及不同图的划分和类别
- 用例图

Q&A